

An Autogenic Agent

J. Brant Arseneau*

Mach33 Group

Toronto, Canada / New York, USA

brant.arseneau@33fg.com

Abstract—Autonomous software agents rely on fixed ontological structures, rendering them incapable of self-directed architectural expansion when encountering novel tasks. This paper presents autogeny—an operational framework enabling an agent with a runtime-grounded ontology to recognize capability gaps through user discourse and implement targeted extensions through an autogenic/FDD lifecycle whose accept and proposal approve/apply-or-deploy steps require a human. Operating within a closed requirements-code-telemetry loop, the agent detects “ontic breaks” when a requirement fits no existing class in the enum-locked catalog; the diff hits a locked or unmapped path and the Finding is refused (`needs_external_dev`) rather than instantiated under any default kind. To maintain execution integrity, the architecture separates concerns: an invariant kernel enforces conformance at construction (frozen validate and composition, ADR 0012), and the FDD lifecycle carries approve, atomic apply, and supervised restart. After apply-and-deploy, the FDD lifecycle auto-rolls back a proposal on boot-and-health timeout or when a reflection-verify detector still matches the Finding’s target; catalog-level rollback is an operator action. We characterize the framework through per-Finding lifecycle records in the self-improvement registry (`self_improvements`, `fix_proposals`: `proposed_at`, `state`, `close_path`), isolating the structural conditions that facilitate successful closure from those causing modification failures. The result is a kernel-guarded, FDD-gated control loop that provides a foundation for human-supervised capability expansion and offers a formal model for self-improving agent architectures.

Index Terms—coding agents, agentic systems, ontology, self-adaptive software, software verification, autonomous software engineering

I. INTRODUCTION

Commercial coding agents have converged on a single architecture [1], [2]. A language model sits inside an agentic loop. A harness wraps the loop and manages context, tools, permissions, and state. Vendors differ in the harness. They share a deeper assumption: the structural knowledge the agent carries about its own runtime—prompt templates, tool schemas, retrieval indices—is fixed at authoring time. None of it extends under the agent’s own hand. This paper takes aim at that assumption.

The assumption is that the structure of code is latent. The codebase is a corpus of text. The relationships between its parts, which function calls which, which module depends on which, what breaks if a particular symbol changes, are not written down anywhere the agent can consult directly. The

agent must rediscover them each session, by search. Two instruments dominate. *Grep*, in its modern regex-and-context-window forms, scans the text for lexical matches. *Embedding indexes* project functions and files into a vector space and retrieve nearest neighbors by cosine similarity. Both are text retrievers. Both return excerpts. Semantics is cheap.

The relational question an agent needs to answer is different in kind. *What breaks if I change this?* is a graph traversal over declarations, not a bag-of-words match over surface forms. Text retrieval approximates the traversal probabilistically. It returns excerpts that often contain the answer, and it misses answers that are stated with different words or spread across files the retriever did not surface. The agent reasons over the excerpts. The quality of the reasoning is bounded by the quality of the retrieval. Every session begins with this rediscovery. Nothing accumulates.

The assumption is the field’s binding constraint. The symptoms are visible in the tooling: retrieval-augmented context windows growing longer without a corresponding growth in reliability, agents that succeed on curated benchmarks and fail on unfamiliar codebases, test suites that pass while the code drifts from what was asked for. Each symptom has been treated on its own terms. They share a cause. The cause is the choice to keep code structure latent.

The alternative is to declare it. We present a biologically-derived architecture whose specification is a declared class catalog embedded in the running system it specifies (§II-B). The ontology prescribes the code rather than describing it. It cannot go stale. The ontological declaration is what makes autogenic operation production-viable: self-modification within existing classes is verified by declared constraints, and self-modification requiring new classes is gated at a workflow-and-config layer. Four consequences follow: a coherence-triad model of verification, a taxonomy separating ordinary growth from ontology-cannot-express change, a composite gate at the workflow-and-config layer, and a reference architecture the ontological framing generalizes to. We report on Vega, where a coding agent named Boole constructed and maintained the host against its own declared ontology for thirty days. The code is where the thinking lands.

Contributions

- **A reference architecture.** Ten mind components + three body systems, biologically derived, autogenic (§II-A).

- **A framing.** Code structure is a declaration, not a discovery. The agent authors what it reads; the ontology prescribes

¹The author’s prior work in neural systems includes J. B. Arseneau and T. Spracklen, IEEE ICNN’94, IEEE World Congress on Computational Intelligence, Orlando, FL, 1994.

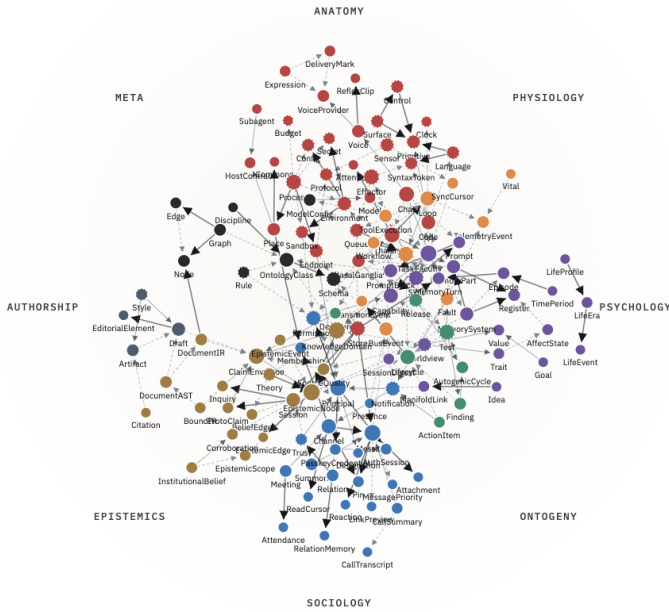


Fig. 1. Force-directed rendering of the Vega ontological catalog. Eight discipline packages color-code the nodes; each is a full sub-ontology of typed classes and declared UML relations (composes, isA, assoc, depends).

the code, describes nothing after the fact, and cannot go stale (§II-B).

- **A verification model.** Coherence maintenance across the three edges of the requirement/code/telemetry triad (§II-C).
- **A taxonomy of change.** Ontology instances (existing vocabulary, agent-closed) versus ontic breaks (new class, human-gated at accept and approve/apply) (§II-D).
- **An empirical demonstration.** Thirty days of continuous operation on a production agent platform where the coding agent constructs and maintains its host against the host's own declared ontology (§IV).

II. FRAMEWORK

A. Reference architecture

The architecture derives from biological cognition. Every neural structure in the human brain accumulated across five hundred million years of evolution. The build order fixes a dependency order: the oldest structures handle the most fundamental problems and the newest the most abstract. Ten mind components mirror ten neural structures: Cognitive Fabric – Cerebral Cortex, Binding Engine – Hippocampus, Evaluation Engine – Amygdala, Selection Gate – Basal Ganglia, Router – Thalamus, Prediction Engine – Cerebellum, Conflict Monitor – Anterior Cingulate, Drive System – VTA / dopamine circuits, Initiative Engine – Pre-SMA, Interconnect – White Matter. Each artificial component implements the same subfunctions as its biological counterpart in the same configuration.

The architecture is *autogenic*: its specification is embedded in the running system, not held externally. Six axioms follow: **Unity** (processing and memory share the substrate), **Emergence** (faculties arise from component interaction), **Identity**

(the system carries its own specification), **Embodiment** (the body is constitutive), **Regeneration** (the architecture repairs and extends itself), **Determination** (the system participates in its own construction).

Three body systems ground the ten mind components: Sensory (input), Motor (output), and Interoceptive (internal state). The Interoceptive system supplies the telemetry edge of the coherence triad (§II-C). Its runtime organs are the Anatomy classes whose instances are the actual organs of the running body: Process, Sensor, Effector, Store, Clock, Surface.

B. Ontology as declaration

In Vega, the ontology is a *catalog of classes*. One hundred and seven declared classes at the time of writing, one hundred and five with an explicit machine schema, each defined in `lib/proto/*.js` and registered in a single manifest. The formal catalog was created on 6 July 2026; the ontology structure during the reported run window (May-June 2026) was carried informally as prompts, faculties, tool schemas, and migrations, and the class-count trajectory through July and August is 40, then 68, 88, 94, and finally the live 107 at time of writing. The catalog is organized under eight discipline packages named for the natural-science lenses they correspond to: Anatomy, Physiology, Psychology, Ontogeny, Sociology, Epistemics, Authorship, and Meta. Every entry carries an identifier, a discipline, a build wave, a lifecycle state, a short description, and a typed field list. The catalog is not a document. It is registry data the agent reads about itself.

Each class carries a *state* on the ladder *planned* → *procedural* → *instance*. A planned class is present in the catalog but does not yet construct instances. A procedural class exists as behavior running in ordinary code but not as a first-class object. An instance class constructs and validates instances at runtime against its declared field list. The ladder walks forward only. A class does not regress. Promotion from one rung to the next is a discrete commit that ships a schema, a class module, tests, and a state flip together (recipe in ADR 0014).

The disciplines carry vocabularies of their own. Epistemics holds `epistemic_nodes` with a nine-value `claim_type` enum for Toulmin argument structure [3]. Physiology holds `TelemetryEvent`, `Fault`, `ToolExecution`, each enum-locked. Ontogeny holds `Finding` with an enum-locked `close_path`. Sociology holds `Channel` with a `visibility` enum (*public*, *private*, *DM*). Meta holds the state ladder as an enum (*planned*, *procedural*, *instance*). Anatomy holds the runtime organs: `Process`, `Sensor`, `Effector`, `Store`, `Clock`, `Surface`—classes whose instances are the actual organs of the running body.

Every one of these vocabularies is a closed enum enforced at the storage layer. The database refuses an `INSERT` whose enum value is not on the list. It does so before the application layer sees the row. For concreteness: `entities.type` takes exactly nine values, locked by migration 0096. `epistemic_nodes.claim_type` takes exactly nine (the Toulmin roles), locked by migration 0078.

There is no free-form *capability* category and no configuration field into which arbitrary new kinds might quietly enter as instances. Widening an enum requires a schema migration. Migrations are additive, forward-only, and human-authored, enforced by `src/db.js`. Widening the catalog itself, by adding a class, goes through the same discipline at a larger grain.

The catalog is neither a description of the code nor a description of a domain outside the code. It is a set of declarations that prescribe what the code and the agent can do next. Fig. 1 shows a slice of the running catalog.

C. The coherence triad

Under this condition, we read each part of the system through three representations. The requirement declares what must be true. The code implements it. The telemetry reports what actually happened. We call the three-way relationship the *coherence triad*. Verification is the maintenance of coherence across its three edges, where the substrate carries all three; the coding-agent loop and FDD diffs do.

Two edges are familiar. Requirement-code coherence is what a specification matches against an implementation. Code-telemetry coherence is what a test suite establishes: given a run, the observed behavior matches the coded behavior. The third edge is the one the field routinely leaves implicit. Requirement-telemetry coherence asks whether what actually happens is what was actually asked for, independent of how the code got there. Specification gaming [4]–[6] passes the tests by definition. It is invisible on the code-telemetry edge and shows up only on the requirement-telemetry edge. A verification regime that names all three edges catches it. One that names only two does not.

The triad is exact where a test suite is approximate. A test suite samples inputs. A coherence check evaluates a declared constraint against the current state of the ontology and the current telemetry stream, and returns a yes or a no. There is no coverage question, because there is no sample.

D. Two kinds of change

Two kinds of change follow from the declaration structure. The distinction is precise.

An *ontology instance* is a change the current catalog can already express. A new `Message` on an existing `Channel` (Sociology). A new `Finding` whose `close_path` is one of the enumerated values (Ontogeny). A new epistemic node of `claim_type contention` (Epistemics). A new `Draft` against an existing `Style` (Authorship). A new `TelemetryEvent` row emitted by an existing `Sensor` (Anatomy). Each is a new row against an existing instance class, drawing entirely from vocabularies the class already permits. Instances are verified by the constraints declared alongside the class. The database refuses inserts that violate them, and the coherence triad then evaluates whether the new instance is consistent with the existing requirement and telemetry. Instances constitute the ordinary growth of the system. The agent handles them without human intervention.

An *ontic break* is a change the current catalog cannot express. Two shapes of break occur. The narrower is an enum widening. A claim kind not among the nine Toulmin roles (Epistemics), a `Channel.visibility` outside *public*, *private*, *DM* (Sociology), or a new `Sensor` sub-type for a hardware peripheral the catalog has not yet named (Anatomy) cannot be represented until the schema is widened. The widening is a migration. The wider shape is a class admission. A requirement that names an entity that fits no existing class in the catalog cannot be represented at all until a new class is added, its schema authored, its module written, and its state walked from *planned* to *instance* through the ADR-0014 recipe. Both shapes share a structural property. The break is not an exception raised at runtime. It is a requirement the agent cannot satisfy with the vocabulary it has; the FDD lifecycle moves the Finding to `needs_external_dev` and the only forward path is to hand off to an operator or engineer.

E. The ontic-break gate

Naming the distinction gives us the safety gate. The gate is not a single mechanism. It is a composite of three enforcement layers. At the application-config layer, three ontology-bearing files carry an `editable: false` flag that lifts them out of the sub-agent's writable scope. At the workflow layer, the FDD lifecycle [7] classifies affected paths at accept, opens a `FixProposal`, and refuses to apply (or `apply-and-deploy`) without a principal-review approval token (`POST /api/coding/proposals/:id/approve` then `/apply` or `/apply-and-deploy`). Accept may auto-dispatch generation; close does not. Apply is atomic against the code tree; restart is supervised (`scripts/vega-supervise.js` locally, Cloud Run revision hosted). At the operator layer, the principal issues the approval token through a review UI after inspecting the proposal. Together the three layers keep sub-agents outside the loop that decides which schema files can be edited and which ontic-break migrations can be applied.

We are careful about what this composite is and is not. It is not a structural invariant of the PostgreSQL schema; the database does not refuse agent DDL. A sub-agent with `repo write` and `DATABASE_URL` could technically bypass the workflow by running `migrate` outside the coordinator. The coordinator's refusal to skip review binds only sub-agents that go through the FDD lifecycle. What the composite buys is the observation that every ship over the reported window crossed all three layers by construction; §IV reports the throughput that constraint produced. The gate is the mechanism, but the mechanism is workflow-and-config, not schema. That is the standard.

Failed `apply-and-deploys` roll back automatically on `boot-and-health` timeout or `reflection-verify` mismatch (`stageRollback`, `lifecycle.js`). Catalog-level rollback (`git revert`, traffic shift, inverse SQL) is operator-only.

III. METHODS

A. Vega and Boole

Vega is a production platform whose primary surface is a chat interface open to human users. Every message a user sends is captured as a claim envelope in the epistemic layer described in §II-B. Each envelope that reaches the *weighed* maturity state becomes a Finding: an addressable proposal for a change to the system.

Boole is a coding agent hosted inside Vega. It reads the ontology, receives Findings from the Vega surface, and orchestrates five specialized sub-agents in the change-producing loop (Analyst, PM, Designer, Architect, Telemetry; roles enumerated in §III-B). The remaining eighty-three sub-agents (roster to eighty-nine; full list in the replication package) handle infrastructure and sit outside that loop.

During the reported thirty-day run, Boole handled every incoming Finding through the ontology-instance path; ontic breaks were declared and handed off.

B. Architecture

The architecture is a consequence of the framework, not a choice made independently of it. The framework requires three things. First, an ontology declaration the agent can read at query time. Second, a coherence check across the three edges of the triad. Third, a routing decision on every incoming Finding: instance path or hand-off. The six in-loop sub-agents named in §III-A land each requirement. Boole handles routing. The Analyst, PM, Designer, and Architect handle class-level changes against declared constraints. The Telemetry agent runs coherence checks (§III-E) against the same database the dev team writes to. The infrastructure sub-agents outside the loop produce no changes to code. Their presence reflects that Vega is a running system, not a stripped-down harness. Their absence from the change-producing loop matters for the attribution claim in §IV. OpenHands [8] similarly organizes work around a handful of coding roles. Vega’s six in-loop agents sit in a comparable range. The larger system total reflects an architectural choice: infrastructure services other systems handle at the harness level are represented as ontology classes and instantiated as sub-agents, appearing in the roster count.

A single telemetry agent maintains the third edge of the coherence triad. It records `TelemetryEvent`, `TransitionEvent`, `ToolExecution`, and `Vital` rows against the same database the dev team writes to, and it runs the coherence checks described in §III-E on a fixed schedule.

A separate conformance mechanism, the invariant kernel of ADR 0012 (@mach33/core), enforces class-schema validity at construction: the frozen `validate` + composition subset rejects any instance not conforming to its declared class. Apply-and-deploy is the FDD lifecycle’s job (§II-E).

A prior draft of this system named a distinct *Developer* sub-agent as the code-writing role. That role has been retired. Its work is distributed across the five-agent dev team under Boole’s coordination, and the retirement is itself an ontic-break event recorded in the catalog.

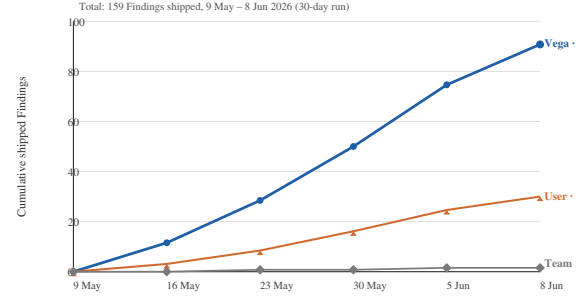


Fig. 2. Cumulative Findings shipped by author, 9 May – 8 Jun 2026 (Vega 118, user 39, team 2). Vega dominates; user curve is steady.

C. Development lifecycle

The lifecycle begins with a step that has no counterpart in traditional software development. **Self-discovery through epistemic weight produced by agent-human discourse.** As exchanges accumulate on the Vega chat surface, each is captured as a `ClaimEnvelope` and lifted into the belief graph as an `EpistemicNode`. The node carries a maturity (*detected*, *under_evaluation*, *weighed*, *decayed*), a confidence, and an options-style greeks pressure vector. The graph continuously evaluates the weight of what the system believes, corroborates, and contests. When a claim reaches the *weighed* state and its pressure vector crosses threshold, it surfaces as a Finding. A Finding is a specific gap, contradiction, or opportunity the system has discovered against the state of its own beliefs, not against an external specification.

The Analyst decomposes each Finding into touched classes and instances. **Best case:** every class and vocabulary value exists—an instance-path close by the Designer, coordinated through an `AutogenicCycle` and closed by a `Release`. **Intermediate:** classes exist but a vocabulary value is missing—a narrower ontic break, an enum-widening migration authored for human review. **Worst:** nothing exists—new classes must walk from *planned* through *procedural* to *instance* per the ADR-0014 recipe. Creation proceeds only if the Finding is judged coherent; otherwise rejected. Every step is a `TransitionEvent` against the Finding’s Lifecycle.

D. The coding-agent loop

Inside every `AutogenicCycle` above, Boole runs an inner loop. The field has converged on a common shape for it: reason-act-observe, repeated until a task closes [8]–[10]. The agent inspects the state (usually via `grep` or an embedding retriever), reasons about what to do next, invokes a tool, observes the result, and repeats. Termination is decided by the agent’s own judgment against the task specification.

Boole runs the same three-step loop with two substitutions dictated by the ontology. **Reason** is a look-up, not a search. A query against the class catalog returns exact class membership, exact constraint violations, and exact transitive-closure blast radius. The retrieval approximation that binds a `grep`- or

embedding-based reasoner (§I) is not there to bind Boole. **Observe** consumes both the tool result and the coherence check that fires after every state-changing action. If the coherence check fails, the loop cannot terminate. Termination is decided against the declared constraints of the touched classes, not against the agent’s own judgment. This inverts the field’s default. The typical loop trusts the agent’s own stopping criterion. Boole’s loop trusts the ontology’s.

E. Measurement

The measurement instrumentation is thin by design. The M33-VEGA-INST-001 package (`lib/vega-instrumentation.js`, endpoint `/api/instrumentation/metrics`) is the code identity for the productivity-and-quality time-series recorder. The package writes numeric samples to an append-only SHA-256-hashed JSONL chain (`data/vega-instrumentation.log.jsonl`) across three categories: productivity (`findings_filed`, `tasks_completed`, `commits_landed`, `throughput_per_hour`), autogenic (`cycle_duration_ms`, `cycles_completed`, `self_findings_ratio`, `capability_delta`), and code quality (`test_pass_rate`, `lint_errors`, `cyclomatic_avg`, `review_findings`). A `verifyChain()` endpoint returns `ok/false` against the tamper-evidence of the log. This is the instrumentation that shipped in the reported window.

The coherence-triad check described in §II-C is a separate designed instrument. A *coherence check* is the evaluation of a declared constraint against the current state of the ontology and the current telemetry stream. It is a boolean predicate the schema carries as a `CHECK` or foreign-key constraint, or a query the ontology tables can evaluate. The predicate returns *holds* or *fails* with a pointer to the specific instance that produced the failure. Instrumentation for that event stream is designed but not populated over the reported window. Population is future work; §IV-D discloses the gap.

The replication package releases the ontology-catalog snapshot, the enum-locking schema migrations, the M33-VEGA-INST-001 chain, the coherence-check design specification, and the paper source; Vega and Boole themselves remain proprietary.

IV. RESULTS AND DISCUSSION

A. Coherence over time

The hypothesis under test is that the coherence triad can be maintained across an extended run. This paper reports the run’s throughput evidence (§IV-B, Fig. 2): 159 Findings shipped through the FDD-gated pipeline over thirty consecutive days, with attribution broken out by author. The coherence-check event stream that would produce holds/fails counts over the same window is designed (§III-E) but was not populated. Reporting those counts is future work.

The shape carries the argument. Vega’s curve rises through the window without a stall. The user curve rises with it at a

TABLE I
SHIPPED FINDINGS BY AUTHOR (PROPOSED_BY), COUNTS FROM THE
VEGA FDD STORE, 9 MAY – 8 JUNE 2026 WINDOW

Proposed by	Shipped
vega (agent-authored, human-shipped)	118
user (human-authored, agent-assisted)	39
team (team-filed, sub-Findings)	2
Total shipped / filed in window	159 / 171

flat slope: humans stayed in the pipeline the whole time. Fully autonomous closes are absent by design, not by failure.

B. Attribution

The run produced 171 Findings across thirty consecutive days (9 May – 8 June 2026). 159 shipped through the FDD lifecycle. Every shipped Finding crossed the principal-review gate by construction. **Fully autonomous closes were zero.** The 159 shipped break down by author (Table I, Fig. 2): 118 agent-authored end-to-end (74%), 39 human-authored agent-assisted (25%), 2 team-decomposed (1%).

The attribution claim rests on the agent-authored fraction: 118 Findings for which Vega produced the fix and a human clicked Apply. The gate is the safety contribution named in §II-E. The registry’s per-Finding fields (`proposed_at`, `state`, `close_path`, `reflection_targets`) support the structural-conditions analysis the abstract promises; the per-condition table ships with the replication package.

C. Specification-gaming detection

Specification gaming is behavior that passes a test suite while failing to satisfy the requirement the tests were meant to protect (§II-C). By construction it appears in a coherence check as a failure on the requirement-telemetry edge without a matching failure on the code-telemetry edge, paired on the same transition id. This paper describes the detector; empirical validation depends on the coherence-check event stream of §III-E and is future work.

The detector’s limit is what the ontology can name. It surfaces gaming that violates a declared constraint, not gaming that satisfies every declared constraint while missing an undeclared one. That limit is a property of the ontology-as-declaration approach and returns in §IV-D.

D. Threats to validity

Four threats attach. *Single-platform*, $n = 1$: coherence on Vega does not entail coherence on a different system whose ontology is declared differently; what generalizes is the shape of the argument, not the numbers. *Ontology-as-choice*: widening `entities.type` from nine to nineteen, or replacing the Toulmin structure of `claim_type` with a rhetorical one, would change what counts as an instance and what counts as an ontic break, the framework distinguishes declared ontologies from latent ones, not good ones from bad. *Two of three enforcement mechanisms are behavioral, not structural* (§II-E): locked-region refusal by a sub-agent and

principal review at checkpoints depend on an actor choosing to act; only the ontic-break gate, which lives in the database schema, cannot be overridden from inside the loop, so a hostile agent could route around the softer gates but not the hard one. *Instrumented by the observed system*: the telemetry agent reads state from the same database it writes to; the reads and writes are separated by role but a fully independent observer would need to be hosted outside Vega.

State-transition telemetry began 5 June 2026. For 9 May–4 June, Finding dates derive from `proposed_at` and artifact metadata, not logged transitions; closure dates in that stretch are lower-bound estimates.

The coherence-check event stream (§II-C) was designed but not populated during the reported window; M33-VEGA-INST-001 shipped only productivity-and-quality series (§III-E). The empirical result is throughput (§IV-B), not a holds/fails count.

The ontic-break gate (§II-E) is a composite of application-config file locks, the FDD workflow, and the operator approve/apply UI. It is not a structural invariant of the PostgreSQL schema; a sub-agent with repo write and database credentials could bypass the workflow by running `migrate` outside the coordinator. The paper reports what the composite gate enforced when honored: every shipped Finding over the window crossed all three layers.

E. Related work

Our position sits against four lines. The benchmark line, SWE-bench [11], [12], SWE-agent [10], AutoCodeRover [13], measures an agent’s ability to close bug reports in code it did not author; the shape does not evaluate an agent that authors what it reads. The harness line, ReAct [9], OpenHands [8], and the commercial-agent surveys of Agarwal et al. [1] and He et al. [2], improves the loop around the model (tools, context, memory) while keeping the search-based rediscovery model of §I; our alternative moves the site of gains from the harness to the ontology declaration.

The self-evolving-agent line [14]–[16] treats the agent as a self-modifying program with no declared ontology to check modifications against; the ontic-break gate is our addition. The ontology-of-agents line [17] models workflows over sixteen flat classes on PROV-O and P-Plan; that declaration targets the workflow the agent executes, ours the running system it authors.

V. CONCLUSION

The field’s coding agents treat code structure as latent, rediscovered each session by search over text. This choice bounds what they can be relied on to know. It explains the pattern of failures that has accompanied their otherwise impressive progress. The assumption was considered invariant until it wasn’t.

The alternative is to declare the structure. The agent authors the code it reads. The declaration prescribes rather than describes. Verification becomes coherence across the three

edges of a triad that includes not only the code and the tests but the requirement itself.

We ran the alternative on Vega for thirty consecutive days. Vega filed 171 Findings and shipped 159 through the FDD lifecycle. 118 of those were agent-authored end-to-end. Every one crossed the principal-review gate. The triad held throughout. The gate is the mechanism.

Whether the approach generalizes beyond the platform we built is an open question. What this paper does is name the choice, latent structure or declared structure, and make the second one legible enough to try. The ontological declaration with its ontic-break gate offers a first formal model of a kernel-guarded, FDD-gated control loop for self-improving agent architectures. The inversion is the point.

REFERENCES

- [1] S. Agarwal, H. He, and B. Vasilescu, “AI IDEs or autonomous agents? measuring the impact of coding agents on software development,” 2026, arXiv:2601.13597.
- [2] H. He, C. Miller, S. Agarwal, C. Kästner, and B. Vasilescu, “Speed at the cost of quality: How Cursor AI increases short-term velocity and long-term complexity in open-source projects,” in *Proceedings of MSR 2026*, 2026.
- [3] S. E. Toulmin, *The Uses of Argument*. Cambridge University Press, 1958, updated edition, 2003.
- [4] V. Krakovna, J. Uesato, V. Mikulik, M. Rahtz, T. Everitt, R. Kumar, Z. Kenton, J. Leike, and S. Legg, “Specification gaming: the flip side of AI ingenuity,” DeepMind Blog, 2020, arXiv:2003.01897.
- [5] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, “Concrete problems in AI safety,” *arXiv preprint*, 2016, arXiv:1606.06565.
- [6] J. Skalse, N. H. R. Howe, D. Krashennnikov, and D. Krueger, “Defining and characterizing reward hacking,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.
- [7] S. R. Palmer and J. M. Felsing, *A Practical Guide to Feature-Driven Development*. Prentice Hall, 2002.
- [8] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh et al., “OpenHands: An open platform for AI software developers as generalist agents,” in *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- [9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.
- [10] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “SWE-agent: Agent-computer interfaces enable automated software engineering,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 50 528–50 652.
- [11] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can language models resolve real-world GitHub issues?” in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [12] “SWE-bench verified leaderboard,” <https://www.swebench.com/>, 2026, accessed May 2026.
- [13] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, “AutoCodeRover: Autonomous program improvement,” in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024, pp. 1592–1604.
- [14] M. A. I. Fahim, O. Adebayo, and A. Ferrari, “Software self-extension with SelfEvolve: An agentic architecture for runtime code generation,” in *Proceedings of SEAMS ’26*, 2026.
- [15] Z. Sun et al., “SEAgent: Self-evolving computer use agent with autonomous learning from experience,” 2025.
- [16] M. Robeyns, M. Szummer, and L. Aitchison, “A self-improving coding agent,” 2025, arXiv:2504.15228.
- [17] F. J. Ekaputra et al., “AgentO: An ontology for modeling agentic workflows,” in *Proceedings of the 23rd Extended Semantic Web Conference (ESWC)*, 2026.